Transcript

Week 8: Supervised Learning Classification- Part 2

Video 1 – Supervised Learning – Classification Part 2

How do machines learn to separate complex data with precision? Support Vector Machines create the optimal boundary, making them powerful for tasks like image recognition and fraud detection.

A Support Vector Machine, or SVM, is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal boundary, known as a **hyperplane**, that best separates different classes in a dataset.

By maximising the margin between data points, SVM ensures accurate classification, even for new, unseen data. This makes it highly effective in applications such as image recognition, spam filtering, and medical diagnosis.

The decision boundary, or hyperplane, is the dividing line in an SVM model that separates different classes in the data. It is chosen to maximise the margin—the distance from the closest data points, known as support vectors. These support vectors are critical as they determine the position and orientation of the hyperplane. Changing them would alter the boundary, making them the most influential points in classification. The maximal margin classifier is the hyperplane that ensures the widest possible separation between classes, enhancing the model's ability to generalize to unseen data. By maximizing this margin, S V M creates a more robust and effective classifier.

This visualization illustrates the maximal margin classifier in a Support Vector Machine for binary classification. The model aims to classify two distinct classes, represented in red and blue, based on two features.

In real-world applications like image recognition and text classification, effective separation of data is crucial, and SVM's excel by not only classifying data but also maximizing the margin between classes.

The decision boundary, shown as a central line, separates the two classes, while the shaded margins indicate the maximum separation achieved by the model. The support vectors, highlighted in yellow, are the closest data points to the boundary and play a critical role in defining its position and orientation. Removing any of these support vectors would alter the decision boundary.

By maximizing the margin and relying on support vectors, SVM's improve classification accuracy and ensure robustness in real-world scenarios.

In a binary classification problem, the objective of a Support Vector Machine is to identify a hyperplane that effectively separates two classes in an n-dimensional feature space. The hyperplane acts as a decision boundary, ensuring that data points on one side belong to one class, while those on the other side belong to a different class.

Mathematically, the hyperplane is represented by the equation:

w dot x plus b equals 0

In this equation, **w** represents the weight vector, which is perpendicular to the hyperplane. The term **x** denotes the input feature vector, while **b** is the bias term, which adjusts the position of the hyperplane. Together, these components determine the orientation and placement of the boundary, allowing the S V M to classify data points into their respective classes.

The primary objective of a Support Vector Machine is not just to find any hyperplane but to identify the one that maximises the margin between two classes. The margin is defined as the distance from the hyperplane to the nearest data points of each class, which are known as support vectors. These support vectors play a crucial role in determining the position and orientation of the decision boundary.

Maximising the margin is essential for improving the robustness and accuracy of the classifier. A larger margin means the model can generalise better to new, unseen data, making it less sensitive to small variations in the dataset. Mathematically, this margin M is maximised by minimising ‖w‖, the Euclidean norm of the weight vector, while ensuring all data points are correctly classified.

By optimising the margin, SVM's create a well-defined decision boundary, enhancing their effectiveness in tasks such as image recognition, text classification, and medical diagnostics.

The optimisation problem in a Support Vector Machine focuses on maximising the margin between the hyperplane and the nearest data points while ensuring correct classification. This is achieved by minimising the squared Euclidean norm of the weight vector w, which helps define the optimal hyperplane.

Mathematically, this is formulated as:

Minimise one-half times the squared norm of **w.**

This expression ensures a robust separation between classes by keeping the decision boundary as far as possible from the closest data points. The factor one-half is included for mathematical convenience during the optimisation process.

The constraint for this optimisation is:

y sub i times the quantity w dot x sub i plus b is greater than or equal to one, for all i

Here, y sub i represents the label of the i-th data point, which can be either plus one or minus one, indicating its class. This condition ensures that each data point is correctly classified with a margin of at least one unit from the decision boundary.

By solving this optimisation problem, SVM constructs a well-defined decision boundary that can effectively generalise to new data, making it a powerful tool for classification tasks.

In this visualisation, we demonstrate the **kernel trick** in Support Vector Machines (SVM) by comparing two kernel functions: a **linear kernel** and a **Radial Basis Function or R B F kernel**.

When solving classification problems, data points may be distributed in complex patterns, making it difficult to find an effective decision boundary. This visualisation compares how different kernels impact classification accuracy.

Left Plot shows SVM with Linear Kernel
On the left, we apply a **linear kernel** to classify data points arranged in interleaved circles. The linear kernel attempts to draw a straight-line boundary, but due to the circular distribution of the data, misclassification occurs. The shaded region highlights areas where the model fails to correctly separate the two classes.

Right Plot shows SVM with R B F Kernel
On the right, we use an **R B F kernel**, which allows for a more flexible decision boundary. The RBF kernel transforms the data into a higher-dimensional space, making linear separation possible. As a result, the decision boundary curves around the data points, significantly improving classification accuracy. The shaded region shows a clear separation between the two classes, demonstrating the effectiveness of the kernel trick.

This comparison highlights the importance of selecting the right kernel for classification tasks. While a linear kernel struggles with complex distributions, the R B F kernel enhances SVM's ability to handle non-linear patterns, leading to improved performance.

Support Vector Machines use **kernel functions** to handle non-linearly separable data by transforming it into a higher-dimensional space where a linear decision boundary can exist.

The **linear kernel** is used when data is already linearly separable. It calculates the dot product between input vectors, measuring their similarity in the original feature space.

The **polynomial kernel** applies a polynomial transformation to the dot product of input vectors. This allows for curved decision boundaries, making it useful for more complex classification tasks. The parameters control the degree of the polynomial and the curve's shape.

The **RBF kernel** is highly effective for non-linear data. It measures the distance between input vectors and applies an exponential decay based on a parameter, allowing for flexible decision boundaries that adapt to the data structure.

By using these kernel functions, SVM can efficiently classify non-linearly distributed data, improving classification performance.

Once the optimal weights are determined from the support vectors, the **SVM decision function** classifies a new data point, x using the equation shown.

In this function, **sign** determines whether the result is positive or negative, which dictates the class to which the new data point belongs.

If the result is **positive**, the data point is classified into one class.

If the result is **negative**, it belongs to the other class.

This decision function effectively uses the **learned hyperplane** to make predictions for new instances, ensuring accurate classification.

The C parameter, also known as the regularisation parameter, plays a crucial role in Support Vector Machines. It controls the trade-off between maximising the margin and minimising classification error.

A high C value prioritises correct classification of all training points. While this leads to high training accuracy, it also increases the risk of overfitting, where the model captures noise instead of the underlying pattern.

Conversely, a low C value allows some misclassifications to achieve a wider margin. This improves the model's generalisation to new data but comes with a higher risk of underfitting, where the model oversimplifies patterns and fails to capture data complexities.

Finding the right balance for C ensures that the SVM model performs optimally, striking the right trade-off between bias and variance.

The **kernel function** in SVM determines how data is mapped into a higher-dimensional space, enabling the algorithm to find a suitable **separating hyperplane**.

There are several types of kernel functions. The **linear kernel** applies no transformation, making it suitable for linearly separable data. The **polynomial kernel** introduces a polynomial transformation, allowing the model to capture **non-linear relationships**, with complexity controlled by the **degree parameter**. The **Radial Basis Function kernel** is commonly used for **non-linear data** and is influenced by the **gamma parameter**. The **sigmoid kernel**, similar to neural networks, maps data using a sigmoid function but is less frequently used.

The **gamma parameter** defines the **influence of a single training example** on the model. A **high gamma value** restricts this influence to nearby points, increasing the risk of **overfitting** and creating complex decision boundaries. A **low gamma value** allows each training example to have a **broader influence**, leading to **smoother decision boundaries**, but it can result in **underfitting** if patterns are not captured well.

Selecting the appropriate **kernel type and gamma value** is essential to **optimising SVM performance** across different datasets.

Support Vector Machines are widely used in various fields due to their ability to classify and detect patterns in complex data.

One of their key applications is **image classification**. By mapping pixel values into a high-dimensional feature space, SVMs can effectively distinguish between different objects in images, such as in facial recognition and handwritten digit classification.

In **text classification**, SVM's excel in spam detection, sentiment analysis, and topic categorisation. They classify documents using features like word frequency or term frequency-inverse document frequency, known as TFIDF.

SVM's are also crucial in **bioinformatics**, where they help classify genes and proteins. They can predict gene functions based on expression data or distinguish between cancerous and non-cancerous tissue samples.

In **finance and risk management**, SVM's are applied to credit scoring, fraud detection, and stock price prediction. They analyse transaction data to identify unusual patterns indicating fraudulent activity or assess credit risk based on borrower profiles.

The **healthcare sector** benefits from SVM's in disease diagnosis and patient classification. They can analyse medical imaging data, such as MRI or CT scans, to detect tumours or predict the likelihood of disease.

SVMs are commonly used in **handwriting recognition**, particularly in Optical Character Recognition or O C R applications. They effectively differentiate between various handwritten characters and digits to convert text into a digital format.

Finally, SVM's are valuable in **anomaly detection**. They help identify defective products in manufacturing or detect unusual patterns in network traffic for cybersecurity applications.

By leveraging SVM's, industries can enhance decision-making, improve efficiency, and strengthen security in data-driven environments.

Support Vector Machines offer powerful classification capabilities, but they come with certain limitations.

SVM's can be computationally intensive, especially when dealing with large datasets, leading to longer training times. They are also sensitive to noisy data and outliers, which can impact classification accuracy.

The choice of kernel is crucial, as different kernels yield different results. Poor selection may lead to underfitting or overfitting. Additionally, while SVM's handle moderate datasets well, they struggle with scalability when dealing with large feature sets.

Interpretability is another challenge, particularly with non-linear kernels, making it difficult to understand how the model makes decisions. Overfitting can also occur, especially in high-dimensional spaces, necessitating proper regularisation and parameter tuning.

Furthermore, SVM's are primarily designed for binary classification. While multi-class extensions exist, they add complexity. Finally, parameter tuning, including adjusting the regularisation and kernel parameters, is time-consuming and often requires extensive cross-validation.

While SVM's offer powerful classification capabilities, understanding their limitations helps in making better model choices.

How will you apply this knowledge to improve your machine learning models?

Video 2 - SVM Implementation

How do machines learn to classify data accurately? Support Vector Machines create clear boundaries for decision-making—let's see how to implement them.

To begin, we import essential libraries for data manipulation, numerical operations, and model evaluation. Num Pie and Pandas handle numerical computations and data

manipulation. Matplotlib and Seaborn generate visualisations. Sigh kit-learn provides datasets, model selection, and evaluation tools. These libraries form the foundation for implementing Support Vector Machines in Python.

To implement Support Vector Machines, we first need to load and prepare the dataset. Here, we use the Iris dataset, which contains 150 samples of iris flowers. Each sample has four features: sepal length, sepal width, petal length, and petal width. The target variable represents the species of the flower—Setosa, Versicolor, or Virginica. We then convert this data into a DataFrame for easier visualisation and processing.

To understand the dataset better, we use data visualisation techniques. The pairplot function from Seaborn generates scatter plots for each feature pair, with colours representing different species. This helps in identifying patterns and distinguishing between the classes. The plot provides insights into how well the features separate different flower species, aiding in model training.

This pairplot visualises relationships between different features of the Iris dataset. Each point represents a flower sample, with colours indicating species. The diagonal plots show feature distributions, while scatter plots reveal patterns and separability between species. This analysis helps in understanding how well features differentiate the classes, aiding in model selection.

To train and evaluate the model, we split the dataset into two subsets: 80% for training and 20% for testing. The training set helps the model learn patterns, while the testing set ensures it performs well on unseen data. This step is crucial for assessing model accuracy and preventing overfitting.

Now, we create an instance of the Support Vector Machine classifier with a linear kernel. The linear kernel is chosen because the Iris dataset is generally linearly separable. We then fit the model to the training data, enabling it to learn patterns for classification.

Now that the model is trained, we can generate predictions. The predict method is used to obtain predicted values for both the training and test datasets. Here, y train pred stores predictions for the training data, while y test pred stores predictions for the test data.

To assess model performance, we use a confusion matrix and a classification report. The confusion matrix compares true versus predicted classifications, helping identify model errors. The classification report provides precision, recall, and F 1-score, offering a detailed evaluation of the model's accuracy.

The SVM model performed exceptionally well on both the training and test datasets. For the training data, the model achieved 97% accuracy, with precision scores of 1.00 for Class 0, 0.97 for Class 1, and 0.95 for Class 2. Recall scores were also high, with Class 0 at 1.00, Class 1 at 0.95, and Class 2 at 0.97. The F 1-scores across all classes indicate strong robustness, with minimal misclassifications. On the test dataset, the model achieved 100% accuracy, with precision, recall, and F 1-scores all at 1.00, confirming its reliability. These results demonstrate that the S V M model is well-suited for classifying species in the Iris dataset, ensuring effective distinction between different categories.

This section visualises the decision boundary of the S V M model using the first two features: sepal length and sepal width. A mesh grid is created to plot the decision

boundary, where each region is colour-coded to indicate the predicted class. The visualisation helps illustrate how the SVM model separates different species in the dataset, providing a clear representation of the classification regions.

This visualisation illustrates the decision boundary of the SVM model for the Iris dataset, using sepal length and sepal width as features. The plot is divided into regions, each shaded to indicate a predicted class. The points represent data samples, with different colours marking their actual classifications. This boundary helps demonstrate how the SVM model separates different species based on these two features.

This slide demonstrates the impact of different SVM kernels on decision boundaries. The linear kernel creates a straight decision boundary, effectively separating classes when the data is linearly separable. The polynomial kernel, with degree 3, introduces curvature to better fit more complex data distributions. The radial basis function, or R B F kernel, captures intricate patterns by creating flexible, non-linear decision boundaries. Each kernel adapts to data in unique ways, influencing classification accuracy.

This slide illustrates how tuning the parameter **C** in an SVM model affects decision boundaries. A lower **C** value, such as **C=1**, allows a softer margin, leading to a more generalised model that tolerates misclassifications. As **C** increases to **10**, the boundary becomes more defined, reducing misclassification but increasing sensitivity to noise. At **C=100**, the model prioritises correct classification, creating a highly complex boundary that may overfit the data. Choosing an appropriate **C** value balances flexibility and accuracy.

You've now explored key concepts and techniques in SVM implementation. Applying these insights can help you optimise decision boundaries and improve model performance.

What steps will you take next to fine-tune your SVM models?

Video 3 - Random Forest

Have you ever wondered how machines make highly accurate predictions from complex data? The secret lies in ensemble learning techniques like **Random Forest**, one of the most powerful algorithms in machine learning.

Random Forest is an **ensemble method** that combines multiple decision trees to improve prediction accuracy and control overfitting. For classification tasks, it determines the output by selecting the most common class among the trees. For regression tasks, it averages the predictions of individual trees. By leveraging multiple decision trees, Random Forest significantly enhances reliability and generalization.

This algorithm incorporates **bootstrap aggregating**, or bagging, which trains each tree on a random subset of data. This enhances diversity, ensuring that the trees do not all follow the same patterns or make identical mistakes. Additionally, at each decision split, it selects a random subset of features, reducing correlation between trees and improving generalization.

These features make Random Forest a powerful machine learning tool, offering strong predictive performance while remaining easy to use and interpret.

In this example, we are working with a dataset of house prices that includes key features influencing property values. The dataset contains attributes such as size, location, and the number of bedrooms. 'Size' represents the area of the house in square feet, 'Location' indicates the neighbourhood, and 'Bedrooms' refers to the number of bedrooms in the house. Our target variable is 'Price,' which we aim to predict based on these features.

In the bootstrap sampling step, we randomly select samples from the original dataset with replacement to create multiple subsets, known as bootstrap samples. This technique increases the diversity of training data for each decision tree in the Random Forest model.

For example, we generate three bootstrap subsets from our house price dataset. Each subset contains a mix of original data points, with some appearing multiple times and others being excluded. This variation ensures each decision tree is trained on slightly different data, improving the overall accuracy and robustness of the model.

In this step, we train a separate regression model, such as a Decision Tree, on each bootstrap subset created in the previous step. Each model learns from its respective subset of the data, allowing for diverse representations of the underlying relationships in the dataset. Model 1 is trained on Subset 1, Model 2 is trained on Subset 2, and Model 3 is trained on Subset 3. By training models on different subsets, we capture various patterns and insights from the data. Each model develops its own decision rules based on the specific samples it has seen. This process is key in ensemble learning, forming the basis for combining model outputs to improve predictive performance.

Now, let's consider a new house with a size of 1700 square feet, located in B, with three bedrooms. Each trained model predicts its price based on its learned patterns. Model 1 estimates the price at $330,000, Model 2 predicts $360,000, and Model 3 estimates $340,000.

To determine the final prediction, we take the average of these values. Adding all three predictions and dividing by three gives a final predicted price of $343,333. This averaging method improves accuracy by leveraging multiple models rather than relying on a single one.

In a classification scenario, instead of averaging, the final prediction would be based on the majority vote among the models. The class that receives the most votes would be chosen as the final prediction, ensuring a more reliable outcome.

Random Forest offers several key advantages, making it a powerful and widely used machine-learning algorithm.

First, **robustness**—it performs well across various datasets and is less sensitive to noise and overfitting compared to individual decision trees. This ensures reliable predictions in different scenarios.

Second, **feature importance**—Random Forest helps identify which features contribute most to the predictions. This insight aids in feature selection and improves model refinement.

Third, **versatility**—it can handle both categorical and numerical data, making it suitable for a wide range of applications, from classification to regression.

These advantages make Random Forest a preferred choice for many machine-learning tasks.

While Random Forest is a powerful algorithm, it has some limitations.

First, **complexity**—due to its ensemble nature, Random Forest is less interpretable than single decision trees. Understanding how specific features influence predictions can be challenging, making it less suitable for applications requiring transparency.

Second, **training time**—Random Forest can take longer to train than simpler models, especially when handling large datasets with many trees. This can be a concern when quick deployment is needed.

Third, **memory usage**—it requires more storage because it maintains multiple decision trees. This can be an issue in resource-constrained environments or on devices with limited computational power.

Despite these drawbacks, Random Forest remains a widely preferred choice due to its strong performance and versatility.

Random Forest is a versatile machine learning algorithm with numerous applications. Let's explore four key use cases.

Firstly, in **medical diagnosis**, Random Forest is effectively used to classify diseases based on patient data. By analysing various health metrics and patient history, it aids in making accurate diagnoses and improving treatment plans.

Secondly, in **fraud detection**, particularly in the finance sector, Random Forest identifies fraudulent transactions. By examining patterns in transaction data, it distinguishes between legitimate and suspicious activities, enhancing security measures.

Thirdly, for **customer segmentation**, businesses utilise Random Forest to group customers based on their purchasing behaviour. This enables targeted marketing strategies and personalised customer experiences, ultimately driving sales and customer loyalty.

Finally, in **recommendation systems**, Random Forest suggests products based on user preferences. By analysing historical data on user interactions, it provides tailored recommendations, enhancing user satisfaction and engagement.

These diverse applications showcase the versatility and effectiveness of Random Forest in solving real-world problems across various domains.

How do you think you can use Random Forest to stand out in the job market? Think about it!

Video 4 - Random Forest Implementation

Want to build a career where your data skills truly matter? Learn to implement Random Forest and unlock your potential to solve real-world problems.

To start our Random Forest implementation, we first need to import the necessary Python libraries. This includes Num Pie and Pandas for data manipulation and numerical operations. We'll use Matplotlib and Seaborn for data visualisation. And crucially, sight kit-

learn, or s k learn, provides the tools for machine learning, including datasets, model selection, and evaluation metrics. As you can see on the screen, we import these libraries using the 'import' statements in Python.

Now, let's load our data. We're using the Diabetes dataset, which contains various health measurements of patients. The target variable indicates whether a patient has diabetes or not. We'll load this dataset directly from sigh kit-learn using the datasets dot load function. We then extract the features and target variables into X and y, respectively. We convert the target variable to binary values, where one represents the presence of diabetes, and zero represents its absence. Finally, we create a Pandas DataFrame for better visualisation and add the target variable as a new column called 'diabetes'. As you can see, we then display the first few rows of the DataFrame using the head function.

Now that we've loaded our data, let's visualise it to gain some insights. We'll use the pairplot function from the Seaborn library. This function creates scatter plots for each pair of features in our dataset. Each point is coloured according to the diabetes status, which helps us visualise the separability of the classes. We specify the DataFrame, the 'diabetes' column for the hue, and the 'deep' palette for colours. We also add a title to the plot, 'Diabetes Dataset Pairplot', and then use plt dot show to display the generated visualisation.

Random Forest is a powerful machine learning algorithm that improves accuracy by combining multiple decision trees. First, we split the dataset—eighty percent for training and twenty percent for testing—to ensure the model is evaluated on unseen data. Then, we train a Random Forest classifier with one hundred decision trees, boosting accuracy and reducing overfitting. This method helps balance model complexity and performance, making it a reliable choice for predictive analytics.

Now that the model has been trained, we can generate predictions using the predict method.

First, we **predict the training set labels** to evaluate how well the model fits the data it has seen. Then, we **predict the test set labels**, which helps assess how well the model generalises to new, unseen data.

These predictions will be used to measure the model’s accuracy and performance.

To evaluate the model’s performance, we use the **confusion matrix** and the **classification report**.

The **confusion matrix** compares true and predicted classifications, highlighting where the model makes mistakes.

The **classification report** provides key metrics such as **precision, recall, and F 1-score**, offering a detailed view of the model’s accuracy.

We assess the model on both the **training data**, to check how well it learned, and the **test data**, to measure its generalisation.

In this Random Forest implementation, we assess model performance using a confusion matrix and classification report.

For the training data, the confusion matrix shows perfect classification, with no misclassifications. The classification report confirms this, showing a precision, recall, and F 1-score of 1.00 for both classes, leading to an overall accuracy of 100%.

For the test data, the confusion matrix indicates some misclassifications, with 12 false positives and 13 false negatives. The classification report shows a precision of 0.74 for class 0 and 0.69 for class 1, with an overall accuracy of 72%.

This demonstrates that while the model performs perfectly on training data, it generalises with slightly lower accuracy on test data, highlighting the potential for overfitting.

In this Random Forest implementation, we analyse feature importance to understand which variables contribute most to the model's predictions.

The model assigns importance scores to each feature, ranking them based on their influence. The top-ranking features have the highest impact on the predictions, while lower-ranked features contribute less.

A bar chart visualises these importance scores, helping identify key factors driving the model's decisions. This insight is useful for refining the model and focusing on the most relevant features.

Video 5 - Random Forest Implementation

When should you use a Support Vector Machine, and when is a Random Forest the better choice? Each model has its strengths—understanding them helps you make the right call for your data.

Support Vector Machines and Random Forests are both powerful tools for classification tasks. However, they work in very different ways. S-V-Ms focus on maximising the margin between classes, while Random Forests use multiple decision trees through ensemble learning.
By comparing them, we can better understand when to use each model and which one performs best in different situations.

Support Vector Machines and Random Forests take very different approaches to classification.
S-V-Ms focus on finding the best boundary between classes by maximising the margin. In contrast, Random Forests improve accuracy by combining multiple decision trees.

One key difference lies in preprocessing: S-V-Ms require feature scaling to work effectively, while Random Forests do not.
When it comes to handling non-linear data, S-V-Ms rely on a technique called the kernel trick, whereas Random Forests manage this naturally through their tree-based structure.

Now let's consider interpretability. S-V-Ms are generally harder to interpret, while Random Forests offer more transparency through feature importance scores.

Another important factor is overfitting. S-V-Ms can overfit easily, especially on small datasets. Random Forests reduce this risk by averaging outputs across many trees.

Finally, in terms of training time, S-V-Ms are typically slower—particularly on large datasets. Random Forests tend to train faster, especially on small to medium datasets.

When deciding between a Support Vector Machine and a Random Forest, consider the type of data and your project needs.
Use an S-V-M when you're working with a small to medium-sized dataset, expect a clear margin between classes, and don't need your model to be easily interpretable.

On the other hand, choose a Random Forest when you want fast results with minimal preprocessing, have a mix of numeric and categorical features, or need to understand which features matter most. Each model brings its own strengths—picking the right one depends on what matters most for your task.

To better understand how Support Vector Machines and Random Forests perform, let's compare them using a real dataset.
We'll use a breast cancer dataset, which is a common binary classification problem in medical research.

Both algorithms will be trained on the same data to ensure a fair comparison.
We'll then evaluate their performance using three key metrics: accuracy, a confusion matrix, and a classification report.
This will help us see where each model excels—and where it might fall short.

To get started, we first import the necessary Python libraries for our machine learning demo.

We use NumPy, and Pandas for handling arrays and data frames.
Mat plot lib and Seaborn help us create charts and visualise results.
From scikit-learn, we import tools for loading datasets, splitting data, training models, and evaluating performance.

This includes the Support Vector Classifier, or S-V-C, and the Random Forest Classifier, as well as metrics like accuracy, confusion matrix, and classification report.
We also import the Standard Scaler to prepare our features for training, especially important for Support Vector Machines.

We begin by loading the breast cancer dataset, which is built into the Sigh-kit-learn library.
The data is converted into a DataFrame using Pandas, so it's easier to view and work with.
We then separate the features from the labels using the target column.
Next, we use the function train-underscore-test-underscore-split, to divide the data—seventy-five percent for training and twenty-five percent for testing.

We set the random state to forty-two to make sure the split is consistent every time we run the code.

Before training a Support Vector Machine, we first scale the feature values.
We use the Standard Scaler to make sure all input features have the same range.
We fit the scaler to the training data and use it to transform both the training and test sets.
Next, we create an S-V-M model using S-V-C, which stands for Support Vector Classifier.
Finally, we train the model using the scaled training data by calling the fit function.

To train the Random Forest model, we first initialise it using the Random Forest Classifier function.
Next, we train the model using the training data by calling the fit function.
This step allows the model to learn patterns from the input features and their corresponding labels so it can make predictions on new data.

Now that the Support Vector Machine model is trained, we evaluate how well it performs on both the training and test data.
First, we use the predict function on the scaled training data to get predictions, then print the classification report to measure accuracy, precision, and recall.
We repeat the same process for the test set using the scaled test data.
Comparing these two reports helps us understand if the model generalises well or if it's overfitting.

Now let's look at how our Support Vector Machine performed on both the training and test datasets.
The classification report gives us four key metrics—precision, recall, F-1-score, and support—for each class.
On the training data, we see very high scores across all metrics, suggesting excellent learning.
On the test data, the scores remain high, but slightly lower than the training set.
This means the model generalises well without overfitting.
The F-1-score is especially useful, as it balances precision and recall in a single number.

We now evaluate the Random Forest model using the same process as before.
First, we use the predict function to generate predictions on both the training and test datasets.
Then, we use classification underscore report to assess how well the model performed.
This includes key metrics like precision, recall, and F-1-score.
By comparing results on the training and test sets, we can check how well the model generalises and whether there is any sign of overfitting.

Let's now review how the Random Forest model performs on both the training and test datasets.
On the training set, we see perfect scores—precision, recall, and F-1-score are all one point zero.
This suggests the model fits the training data extremely well.
However, when we look at the test set, the scores are slightly lower.
The F-1-score drops to zero point ninety-six overall, which is still strong but shows a small gap.
This tells us that the model generalises well, though it may be slightly overfitting to the training data.

Comparing train and test metrics helps us understand model behaviour more clearly.

Let's compare how both models performed visually using confusion matrices.
Each square shows how many predictions the model got right or wrong for each class.
On the left, the S-V-M confusion matrix shows 52 correct predictions for class zero and 87 for class one, with only two misclassifications in each class.
On the right, Random Forest predicted slightly less accurately for class zero, with three misclassifications, but matched S-V-M with 87 correct for class one.
Both models perform well, but S-V-M has a slight edge in precision for this dataset.
Confusion matrices give a quick, visual way to assess prediction performance.

To wrap up, let's highlight the key takeaways when choosing between S-V-M and Random Forest models.
First, always evaluate your model on both training and test sets. This helps detect if it's overfitting or underfitting.
Support Vector Machines perform best with clean, well-scaled data and clear class boundaries.
Random Forests, on the other hand, are better at handling noisy or complex datasets.
There's no one-size-fits-all solution—your choice should always depend on the specific use case and the nature of your data.
Understanding each model's strengths will help you make smarter, more informed decisions.

Ready to choose the right model?
Now that you've seen how Support Vector Machines and Random Forests compare, it's time to put your understanding into practice. Try applying both models to a new dataset—and see which one performs better and why.

Video 6 - Hyperparameter Tuning Random Forest with GridSearchCV

Want better results from your Random Forest?
Grid Search C-V helps you find the best settings to boost model performance.
Let's learn how to tune it step by step.

Hyperparameters are settings you define before training a machine learning model.
They control how the model learns—for example, in a Random Forest, you might set the number of trees or the maximum depth of each tree.
These are not learned from the data but are chosen ahead of time.
Tuning them properly can lead to a big improvement in model performance.

Let's explore the key hyperparameters you can tune in a Random Forest model—each one plays a specific role in how the model learns.
We start with n underscore estimators, which sets the number of trees in the forest. More trees often lead to better accuracy, but they also increase training time.

Next is max underscore depth, which controls how deep each tree can grow. Setting this helps manage overfitting.
Then we have min underscore samples underscore split, which defines the minimum number of samples needed to split a node.
Closely related is min underscore samples underscore leaf—this sets the minimum number of samples that must be at a leaf node.

Max underscore features decides how many features the model considers when making a split, affecting both performance and speed.
Finally, bootstrap controls whether sampling with replacement is used when building the trees.
Together, these hyperparameters give you the tools to balance accuracy, training time, and generalisation.

Tuning hyperparameters helps you get the most out of your model.
It allows you to find the best possible performance on your validation data—not just what works by default.

It also helps you avoid common problems like overfitting, where the model memorises the training data, or underfitting, where it struggles to learn anything useful.
And by using tools like Grid Search C-V you can automate the process and experiment more efficiently.

Grid Search cross-validation—is a method for tuning hyperparameters.
It performs an exhaustive search, trying every possible combination of the values you define.
Each combination is evaluated using cross-validation, which helps estimate how well it will perform on unseen data.

At the end, it returns the best combination of hyperparameters based on validation performance.
This makes the tuning process more systematic and reliable.

Before we begin tuning our model, we need to import a few essential libraries.
We'll use pandas and NumPy for working with data.

From scikit-learn, we import the breast cancer dataset and the Random Forest Classifier.
We also bring in tools for splitting the data and performing Grid Search C-V.

Finally, we import classification underscore report to help evaluate the model's performance.

We begin by loading the breast cancer dataset using load underscore breast underscore cancer.
We convert the features into a DataFrame using pandas, and extract the target labels as a separate Series.
Next, we split the data into training and test sets using train underscore test underscore split.
This step ensures we can evaluate the model on unseen data and avoid overfitting.

To start tuning, we define a parameter grid—called param underscore grid—which holds the values we want Grid Search C-V to explore.
We include settings like the number of trees, maximum depth of trees, and the minimum number of samples needed to split or end a node.

Each hyperparameter has multiple options, and Grid Search will try every possible combination.
Here, we have six parameters with different choices, which results in ninety-six combinations for evaluation.

We start by creating a Random Forest classifier and setting a random state for reproducibility.
Next, we initialise Grid Search C-V using the classifier, the parameter grid we defined earlier, and set cross-validation to three folds.

We also use one job for processing and enable detailed logging by setting verbose to two.
Finally, we train the grid search model on the training data using the fit function.

After training with Grid Search C-V, we use two simple commands to extract the results.
The first one, grid underscore search dot best underscore params underscore, gives the best combination of hyperparameters found during the search.

The second one, grid underscore search dot best underscore score underscore, shows the highest cross-validation score achieved using those settings.
This helps us confirm both the configuration and the performance of our tuned model.

Once the Grid Search is complete, we extract the best-performing model using grid underscore search dot best underscore estimator underscore.
Then, we use it to predict outcomes on the test set by calling best underscore model dot predict with X underscore test.

Finally, we assess the model's accuracy using classification underscore report, which compares the predicted values with the true labels.
This confirms how well the optimised model performs on unseen data.

Grid Search C-V helps you go beyond default settings to find the best hyperparameter combinations.
Random Forest is already a strong algorithm, but tuning can make it perform even better.
And always remember—evaluate your model on both the train and test sets to ensure it generalises well.

Ready to put theory into practice?

Try tuning your own Random Forest model using Grid Search C-V and see how much the performance improves. Understanding the "why" behind the settings is what sets you apart.